PrepME AI — Day 6 Notes (MongoDB Relationships & Notes CRUD)

1. Day 6 Goal

Until now we had:

```
User Authentication
 ↓
JWT Authentication
 ↓
Protected Routes
```

Day 6 introduced:

```
Real Application Data
```

We started building a Notes application where:

```
User
 ↓
Creates Notes
 ↓
Views Own Notes
```

This is how most real applications work.

Examples:

```
User → Orders
User → Posts
User → Tasks
User → Documents
User → Notes
```

---

2. CRUD Overview

CRUD stands for:

```
C = Create
R = Read
U = Update
D = Delete
```

Examples:

```
Create User
Read User
Update User
Delete User
```

---

3. CRUD Learned So Far

CREATE

```
save()
```

Used for:

```
Creating Users
Creating Notes
```

READ

```
findOne()
```

Used for:

```
Finding User By Email
```

```
find()
```

Used for:

```
Getting Multiple Notes
```

UPDATE

Not implemented yet.

Will learn:

```
findByIdAndUpdate()
```

DELETE

Not implemented yet.

Will learn:

```
findByIdAndDelete()
```

---

4. Database Relationships

New concept introduced:

```
Relationships
```

Example:

User Collection

```
{
  "_id": "111",
```

```
  "email": "user@gmail.com"
}
```

Notes Collection

```
{
  "_id": "222",
  "title": "Learn JWT",
  "user": "111"
}
```

Notice:

```
user = 111
```

This creates a relationship between:

```
Note
 ↓
User
```

---

5. Why Use User ID Instead Of Email?

Bad:

```
{
  "user": "abc@gmail.com"
}
```

Good:

```
{
  "user": "685f123..."
}
```

Reason:

```
Emails can change.
User IDs do not.
```

Most database relationships use IDs.

---

6. Note Model

Created:

```
models/Note.js
```

Schema:

```
const noteSchema = new mongoose.Schema({

  title: {
    type: String,
    required: true
  },

  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User"
  }

})
```

---

7. Understanding ObjectId

Example:

```
mongoose.Schema.Types.ObjectId
```

Meaning:

```
Store MongoDB document ID
```

Example value:

```
685f7e9c2a4d...
```

Used for relationships.

---

8. Understanding ref

Example:

```
ref: "User"
```

Meaning:

```
This field references
the User collection
```

This allows MongoDB to connect:

```
Notes ↔ Users
```

Later we'll use this feature with:

```
populate()
```

---

9. JWT Recap

During login:

```
jwt.sign({
  userId: user._id,
```

```
  email: user.email
})
```

JWT contains:

```
{
  "userId": "111",
  "email": "abc@gmail.com"
}
```

---

10. JWT Verification Flow

Request:

```
Authorization: TOKEN
```

↓

Middleware:

```
jwt.verify()
```

↓

Returns:

```
decoded
```

↓

```
req.user = decoded
```

Result:

```
req.user.userId
req.user.email
```

available inside controllers.

---

11. Understanding req.user

Suppose:

```
decoded = {
  userId: "999",
  email: "abc@gmail.com"
}
```

Middleware:

```
req.user = decoded
```

Now:

```
req.user.userId
```

returns:

```
999
```

and

```
req.user.email
```

returns:

```
abc@gmail.com
```

---

12. Create Note API

Created:

```
POST /api/users/notes
```

Purpose:

```
Create new note
for logged-in user
```

---

13. Create Note Controller

```
const note = new Note({

  title: req.body.title,

  user: req.user.userId

})
```

Example

Incoming Request:

```
{
  "title": "Learn JWT"
}
```

Logged-in User:

```
req.user.userId = "999"
```

Stored Document:

```
{
  "title": "Learn JWT",
  "user": "999"
}
```

---

14. Why JWT Matters

Without JWT:

```
Who is creating the note?
```

Backend cannot know.

With JWT:

```
JWT
 ↓
Middleware
 ↓
req.user.userId
 ↓
Save Note
```

Backend always knows the current user.

---

15. Notes Collection

MongoDB automatically created:

```
notes
```

collection.

Example document:

```
{
  "_id": "...",
  "title": "Learn JWT",
  "user": "685f..."
}
```

---

16. Reading Multiple Documents

New method learned:

```
find()
```

Purpose:

```
Get multiple documents
```

Example:

```
await Note.find()
```

returns:

```
[
  {...},
  {...},
  {...}
]
```

Array of documents.

---

17. Difference Between find() and findOne()

findOne()

Returns:

```
{
  ...
}
```

Single document.

---

find()

Returns:

```
[
  {...},
  {...}
]
```

Array of documents.

---

18. Get Notes API

Created:

```
GET /api/users/notes
```

Purpose:

```
Return all notes
belonging to logged-in user
```

---

19. Get Notes Controller

```
const notes =
  await Note.find({

    user:
      req.user.userId
```

```
  })
```

Meaning:

```
Find all notes
where user field matches
logged-in user's ID
```

---

20. Example Query

Suppose:

```
req.user.userId = "111"
```

MongoDB executes:

```
Note.find({
  user: "111"
})
```

Database:

```
{
  "title": "JWT",
  "user": "111"
}
```

```
{
  "title": "React",
  "user": "111"
}
```

```
{
  "title": "DSA",
```

```
  "user": "222"
}
```

Response:

```
[
  {
    "title": "JWT"
  },
  {
    "title": "React"
  }
]
```

Only matching notes returned.

---

21. Why Frontend Does Not Send User ID

Frontend sends:

```
JWT Token
```

NOT:

```
User ID
```

Backend already knows user:

```
JWT
 ↓
verify()
 ↓
decoded
 ↓
req.user
 ↓
MongoDB Query
```

This is how most modern applications work.

---

22. Route Design

Interesting observation:

POST

```
/api/users/notes
```

creates note.

GET

```
/api/users/notes
```

reads notes.

Same endpoint.

Different HTTP method.

This follows REST API design principles.

---

23. Request Flow For Create Note

```
Frontend
 ↓
JWT Token
 ↓
Auth Middleware
 ↓
Verify Token
 ↓
req.user
 ↓
Create Note
```

```
 ↓
MongoDB
```

---

24. Request Flow For Get Notes

```
Frontend
 ↓
JWT Token
 ↓
Auth Middleware
 ↓
Verify Token
 ↓
req.user.userId
 ↓
MongoDB Query
 ↓
Return User Notes
```

---

25. Most Important Learning Today

Authentication tells backend:

```
WHO the user is
```

Database queries determine:

```
WHAT data belongs to them
```

Example:

```
JWT
 ↓
User ID
 ↓
MongoDB Query
```

```
  ↓
User-Specific Data
```

This is one of the most important backend concepts.

---

26. Concepts Learned So Far

✅ Express Routes

✅ Controllers

✅ MongoDB Models

✅ Environment Variables

✅ Password Hashing (bcrypt)

✅ JWT Authentication

✅ Middleware

✅ Protected Routes

✅ localStorage

✅ MongoDB Relationships

✅ CRUD (Create & Read)

✅ User-Specific Data Retrieval

---

27. Next Learning Goals (Day 7)

Coming next:

```
Delete Note
Ownership Validation
Authorization
findById()
```

```
findByIdAndDelete()
Frontend Notes UI
```

Important security concept:

```
User A must not be able
to delete User B's note
```

This introduces Authorization, which is different from Authentication.

Key Takeaway

```
JWT tells backend WHO the user is.

MongoDB queries decide
WHAT data the user can access.
```